HIGH-LEVEL DESIGN

RAG-Based Customer Support Assistant
WITH LANGGRAPH & HUMAN-IN-THE-LOOP

Internship Project Document
Version 1.0 | System Architecture & Design
Document Type: High-Level Design (HLD)
System: RAG Customer Support Assistant
Technology Stack: LangGraph, ChromaDB, Ollama, Python
Design Phase: Architecture & System Overview

---

1. System Overview

1.1 Problem Definition

Modern customer support systems face challenges such as handling large volumes of queries, maintaining response accuracy, and reducing dependency on human agents. Traditional FAQ-based systems fail when queries are phrased differently or require contextual understanding.

Additionally, standalone Large Language Models (LLMs) may produce hallucinated responses when not grounded in real data.

This project addresses these challenges by designing a **Retrieval-Augmented Generation (RAG)** system that retrieves relevant information from a structured PDF knowledge base and generates responses grounded in actual content.

The system also introduces **Human-in-the-Loop (HITL)** capability, ensuring that queries with low confidence or missing context are escalated to a human agent.

1.2 Scope of the System

IN SCOPE

- Single PDF knowledge base ingestion
- Semantic retrieval using ChromaDB embeddings
- LangGraph-based workflow orchestration
- Conditional routing (Answer / Escalate)
- Human-in-the-Loop escalation via CLI
- Confidence-based decision making

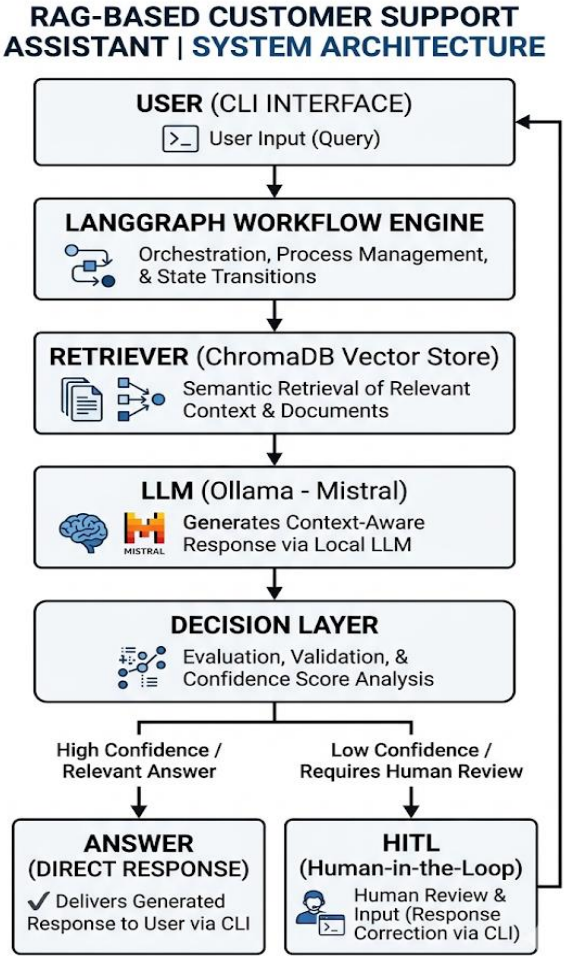
OUT OF SCOPE

- Multi-tenant architecture
- Real-time document updates
- Web API deployment (FastAPI)
- Distributed vector databases
- Sentiment analysis
- Voice-based interfaces
- Redis queue-based HITL system

---

2. Architecture Diagram

The system is organized into multiple logical layers representing ingestion and query processing.



Design Principle:
Each layer is modular and can be replaced independently. For example, the LLM (Ollama) can be replaced with cloud-based models without affecting retrieval or workflow logic.

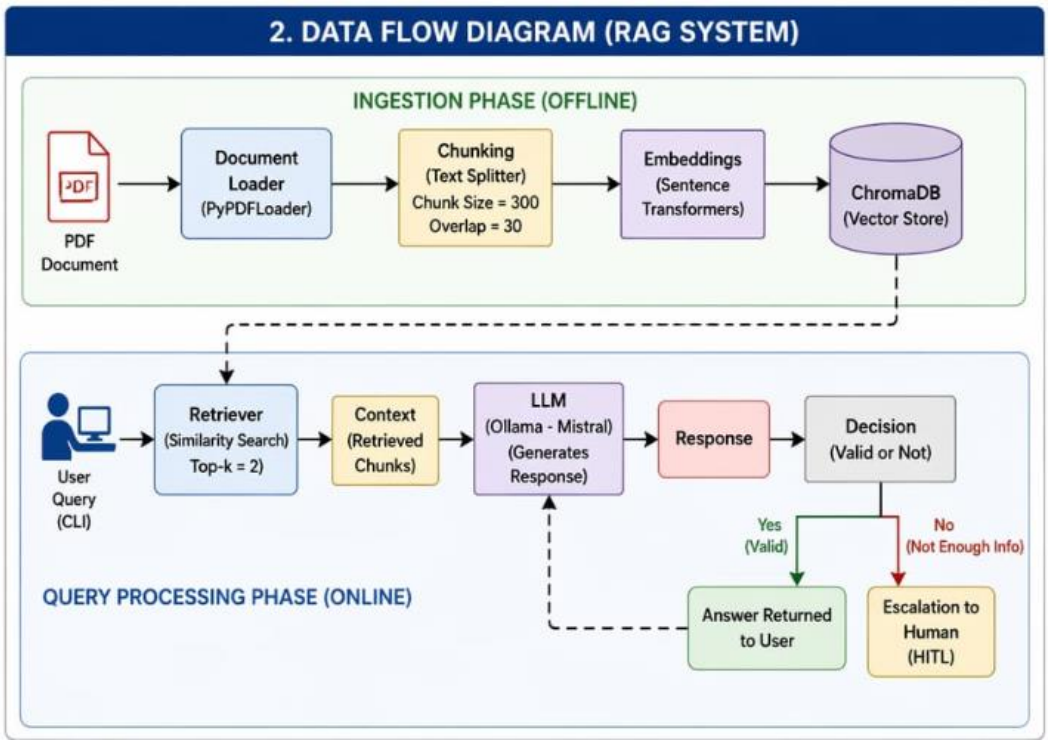
---

3. Component Descriptions

Component	Role	Technology	Output
Document Loader	Reads PDF and extracts text	PyPDFLoader	Raw text
Chunking Module	Splits text into chunks	RecursiveCharacterTextSplitter	Text chunks
Embedding Model	Converts text to vectors	Sentence Transformers	Vector embeddings
Vector Store	Stores embeddings	ChromaDB	Retrieved chunks
Retriever	Fetches relevant chunks	ChromaDB similarity search	Top-k chunks
LLM	Generates responses	Ollama (Mistral)	Answer
LangGraph Engine	Controls workflow	LangGraph	State transitions
Routing Layer	Decides answer vs escalation	Rule-based logic	Decision
HITL Module	Handles escalation	CLI input	Human response

---

4. Data Flow



4.1 Ingestion Pipeline (Offline)

Step 1 — Load:
The PDF is loaded using a document loader and text is extracted.

Step 2 — Chunk:
Text is split into chunks using a sliding window approach:

- Chunk size: 300
- Overlap: 30

Step 3 — Embed:
Each chunk is converted into a vector embedding using a transformer-based embedding model.

Step 4 — Store:
Embeddings are stored in ChromaDB along with metadata.

4.2 Query Lifecycle (Online)

Step 1 — Receive:
User submits a query via CLI.

Step 2 — Retrieve:
Query is embedded and top-k relevant chunks are retrieved.

Step 3 — Generate:
LLM receives query + context and generates a response.

Step 4 — Validate:
System checks response quality using rule-based logic.

Step 5 — Route:

- High confidence → Direct answer
- Low confidence → Escalation

Step 6 — Respond:
Final response is shown (LLM or human).

---

5. Technology Choices

Technology	Category	Why Chosen	Alternatives
ChromaDB	Vector DB	Local, lightweight, easy integration	FAISS, Pinecone
LangGraph	Workflow	Graph-based decision system	LangChain chains
Ollama	LLM	Free, local, no API cost	GPT-4, Gemini

Sentence Transformers	Embeddings	Efficient and accurate	OpenAI embeddings

---

5.1 Why ChromaDB

ChromaDB is lightweight and runs locally, making it ideal for this project. It supports metadata and similarity search without requiring external infrastructure.

5.2 Why LangGraph

Unlike linear pipelines, LangGraph enables conditional routing and state management, which is essential for implementing HITL workflows.

---

6. Scalability Considerations

6.1 Handling Large Documents

- Batch processing of chunks
- Efficient embedding generation
- Chunk-based retrieval

6.2 Increasing Query Load

- Stateless design
- Efficient retrieval using vector search
- Potential extension to API-based deployment

6.3 Latency Considerations

Operation	Approx Time	Optimization
Retrieval	Low	Efficient vector search
LLM Generation	Medium	Prompt optimization
End-to-End	Moderate	Reduce chunk size

Critical Design Decision

The HITL escalation path is intentionally slower, as it prioritizes correctness over speed. Incorrect automated responses are avoided by routing uncertain queries to a human.